

Università degli Studi di Salerno
Dipartimento di Informatica



Documentazione di Progetto

Fortify

AI-Powered Social Engineering
Simulation Framework

Orlando Sica
Mattia Guariglia

Candidati
Mat. 0522502033
Mat. 0522502034

Anno Accademico 2025/2026

Sommario

Fortify è una piattaforma *enterprise* per la simulazione controllata di campagne di *social engineering* (phishing ed *smishing*), progettata per valutare e migliorare il livello di consapevolezza di sicurezza degli utenti all'interno di un'organizzazione. L'idea di base è trasformare l'attacco di ingegneria sociale in un "laboratorio controllato": l'organizzazione definisce il perimetro (aziende, dipendenti-bersaglio, tipologia di campagna), il sistema raccoglie automaticamente informazioni pubbliche sui target tramite tecniche di OSINT, genera con un modello linguistico di grandi dimensioni contenuti malevoli *simulati* e altamente personalizzati, li veicola verso i destinatari e misura il comportamento reale delle persone lungo l'intero ciclo di vita dell'attacco. Al termine, Fortify produce metriche e report strutturati che consentono di identificare i punti deboli e pianificare azioni di formazione mirata.

Il framework è realizzato con uno stack TypeScript moderno e full-stack (Nuxt 4, Drizzle ORM, PostgreSQL), integra la personalizzazione dei contenuti tramite il modello *Google Gemini* orchestrato dal framework Mastra, e sfrutta un motore di *scraping* basato su browser headless per la fase di raccolta OSINT. L'intera architettura è progettata nel rispetto di rigorosi standard etici e di privacy, lavorando esclusivamente con dati pubblici, credenziali simulate e simulazioni controllate prive di reale compromissione.

Questo documento descrive l'architettura del sistema, le scelte tecnologiche, i dettagli implementativi di ciascun componente, la campagna di testing e validazione condotta end-to-end, i risultati ottenuti e le limitazioni note con i relativi sviluppi futuri.

Indice

Sommario	1
1 Introduzione	6
1.1 Contesto e motivazione	6
1.2 Obiettivi del progetto	6
1.3 Considerazioni etiche e legali	6
1.4 Struttura del documento	7
1.5 Nota sul gruppo di lavoro	7
2 Architettura del Sistema	8
2.1 Visione d'insieme	8
2.2 Componenti principali	8
2.3 Component diagram	9
2.4 Sequence diagram	10
2.5 Modello dei dati	10
3 Scelte Tecnologiche	12
3.1 Frontend	12
3.2 Backend	12
3.3 Intelligenza artificiale e scraping	12
3.4 Payload e simulazione di compromissione	13
3.5 DevOps e qualità del codice	13
4 Implementazione	14
4.1 Architettura a livelli del backend	14
4.2 Autenticazione e multi-tenancy	14
4.3 Target Information Collector (OSINT)	14
4.4 Phishing Campaign Generator	15
4.5 NLP Text Personalizer	15
4.6 Campaign Launcher	16
4.7 Interaction Logger	16
4.8 Risk Report Generator e Dashboard	16
5 Testing e Validazione	18
5.1 Ambiente di test	18
5.2 Test end-to-end della campagna e-mail	18
5.3 Test dei singoli moduli e casi di sicurezza	18
5.4 Riepilogo dei risultati	19
6 Valutazione dei Risultati	20

7	Limitazioni e Sviluppi Futuri	21
7.1	Funzionalità non implementate (sviluppi futuri)	21
7.2	Funzionalità implementate con vincoli	21
7.3	Miglioramenti di prodotto	21
8	Conclusioni	22
	Bibliografia	23

Elenco delle figure

2.1	Component diagram: architettura modulare e flussi di dati.	9
2.2	Sequence diagram del flusso operativo end-to-end.	10
4.1	Architettura a livelli del backend.	14

Elenco delle tabelle

2.1	Tabelle principali del modello dati.	11
4.1	Tipi di evento di interazione.	16
5.1	Riepilogo della campagna di test.	19

1 Introduzione

1.1 Contesto e motivazione

La stragrande maggioranza degli incidenti di sicurezza ha origine da un fattore umano: un dipendente che clicca su un link malevolo, apre un allegato o inserisce le proprie credenziali su una pagina contraffatta. Nessuna difesa perimetrale è in grado di neutralizzare completamente questo vettore, perché il bersaglio non è un sistema ma una persona. Per questo motivo la *security awareness* è diventata una componente essenziale della postura di sicurezza di ogni organizzazione, e la simulazione controllata di attacchi di social engineering rappresenta lo strumento più efficace per misurarla.

Le piattaforme tradizionali di *phishing simulation* inviano tuttavia messaggi generici, uguali per tutti i destinatari. Fortify parte dall'osservazione che gli attacchi reali più pericolosi sono quelli *mirati* (spear phishing): messaggi costruiti su misura a partire da informazioni pubbliche relative alla vittima. Il progetto ha quindi l'obiettivo di riprodurre in modo etico questa capacità, combinando raccolta OSINT automatizzata e generazione di contenuti tramite intelligenza artificiale.

1.2 Obiettivi del progetto

Gli obiettivi funzionali del sistema sono:

- raccogliere e normalizzare informazioni pubbliche sui bersagli (LinkedIn, GitHub, siti web personali) in profili strutturati;
- generare campagne di phishing simulato personalizzate per singolo destinatario sfruttando un modello linguistico di grandi dimensioni;
- veicolare le campagne tramite canali multipli (e-mail e SMS) mantenendo la tracciabilità di ogni singola comunicazione;
- tracciare in modo granulare le interazioni degli utenti (apertura, click, inserimento credenziali, esecuzione di allegati);
- produrre report di rischio con metriche, individui ad alto rischio e raccomandazioni formative;
- garantire l'isolamento multi-tenant tra organizzazioni diverse.

1.3 Considerazioni etiche e legali

Fortify è uno strumento a duplice uso e la sua legittimità dipende dal contesto d'impiego. L'intero progetto è stato concepito per l'uso *difensivo* in scenari autorizzati (security awareness aziendale, esercitazioni interne, penetration testing su perimetro concordato). Sono adottate le seguenti garanzie:

- vengono raccolti esclusivamente dati *pubblici*; nessuna informazione personale viene compromessa senza consenso;
- le pagine di login simulate non memorizzano credenziali reali di sistemi di produzione: i dati inseriti sono registrati unicamente a fini di reportistica e di rimozione della vulnerabilità;
- gli allegati con payload sono generati in ambiente controllato e non causano alcun danno reale ai sistemi;
- la portata di ogni campagna è limitata ai soggetti autorizzati definiti dal *Company Admin*.

1.4 Struttura del documento

Il Capitolo 2 descrive l'architettura del sistema e i flussi tra i componenti; il Capitolo 3 motiva le scelte tecnologiche; il Capitolo 4 approfondisce l'implementazione di ciascun modulo; il Capitolo 5 riporta la campagna di testing e validazione end-to-end; il Capitolo 6 discute i risultati; il Capitolo 7 elenca le limitazioni note e gli sviluppi futuri; il Capitolo 8 conclude.

1.5 Nota sul gruppo di lavoro

Il progetto è stato inizialmente pianificato per un gruppo di tre studenti. In seguito al ritiro di un componente, lo sviluppo è proseguito con due membri. Di conseguenza, alcune funzionalità previste in fase di analisi sono state deliberatamente ridotte di portata o rimandate a sviluppi futuri; tali scelte sono documentate esplicitamente nel Capitolo 7.

2 Architettura del Sistema

2.1 Visione d'insieme

Il sistema è organizzato come applicazione full-stack con backend a API REST *stateless* e frontend a dashboard interattiva. Il flusso operativo principale parte da una o più organizzazioni e dai relativi *employee target* inseriti dai *Company Admin*. Ogni target è descritto da un profilo di base (nome, e-mail, ruolo, dipartimento) opzionalmente arricchito da riferimenti a profili pubblici.

La prima fase è la raccolta e l'arricchimento OSINT: un modulo di scraping automatizzato, schedato periodicamente, estrae dati rilevanti dai profili pubblici e li normalizza nel database. Su questa base il Company Admin crea una o più campagne, associando un insieme di target a una tipologia di scenario (password reset, finta fattura, CEO fraud). Al lancio, il backend invoca il modulo di personalizzazione AI che, combinando template, dati OSINT e metadati organizzativi, genera per ciascun destinatario un messaggio credibile e differenziato. Il *Campaign Launcher* veicola i messaggi mantenendo identificatori univoci per link e payload. Durante e dopo il lancio, il *Interaction Logger* registra ogni evento; infine il *Risk Report Generator* aggrega i dati in metriche e report.

2.2 Componenti principali

L'architettura è modulare e organizzata in sette componenti logici.

Dashboard

Punto di ingresso per l'amministratore: consente di configurare organizzazioni, target e campagne e di visualizzare in modo centralizzato metriche e risultati delle simulazioni.

Target Information Collector

Raccoglie dati pubblici tramite tecniche di OSINT (scraping di LinkedIn e GitHub con browser headless, analisi di siti web) e produce una rappresentazione strutturata e storicizzata del target.

Phishing Campaign Generator

Combina i profili con template di campagna predefiniti, selezionando lo scenario più adatto e impostando i *placeholder* che il modello linguistico andrà a risolvere.

NLP Text Personalizer

Prende in input gli "scheletri" dei messaggi e, tramite un modello linguistico di grandi dimensioni, genera testi naturali e contestualizzati per ciascun destinatario, adattando tono, lingua e livello di formalità.

Campaign Launcher

Gestisce la distribuzione dei messaggi tramite i canali configurati (e-mail e SMS), l'assemblaggio dei contenuti con il tracciamento e l'eventuale allegato con payload controllato.

Interaction Logger

Intercetta e registra tutte le azioni compiute dai destinatari (apertura, click, compilazione di form, esecuzione di allegati), associando ogni evento a campagna, target e messaggio.

Risk Report Generator

Elabora i dati prodotti dal logger calcolando indicatori chiave (tasso di apertura, *click-through rate*, *submission rate*), individua i reparti più vulnerabili e produce report esportabili.

2.3 Component diagram

La Figura 2.1 rappresenta i componenti e i flussi di dati principali.

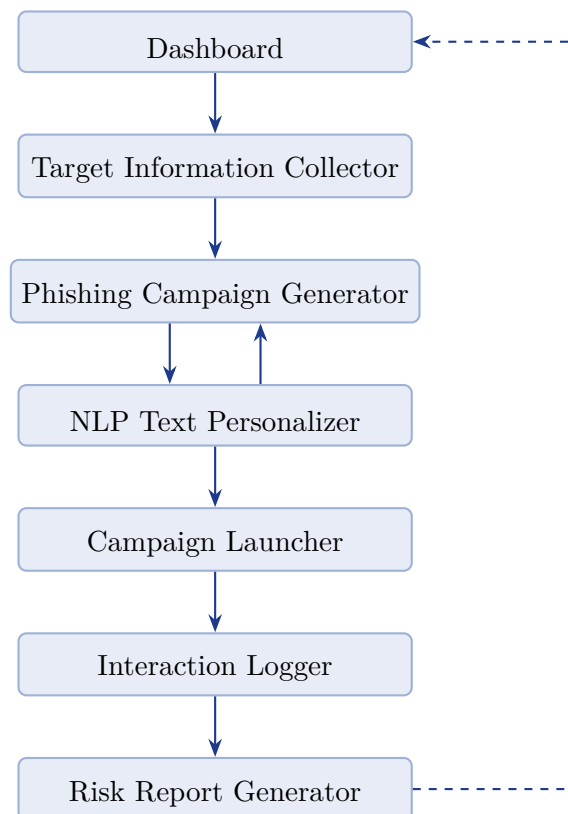


Figura 2.1: Component diagram: architettura modulare e flussi di dati.

Il *NLP Text Personalizer* fornisce inoltre un ciclo di *feedback* verso il *Phishing Campaign Generator* per il raffinamento iterativo dei template, mentre il *Risk Report Generator* chiude il ciclo alimentando la *Dashboard*.

2.4 Sequence diagram

La Figura 2.2 illustra la sequenza temporale di un'esecuzione tipica.

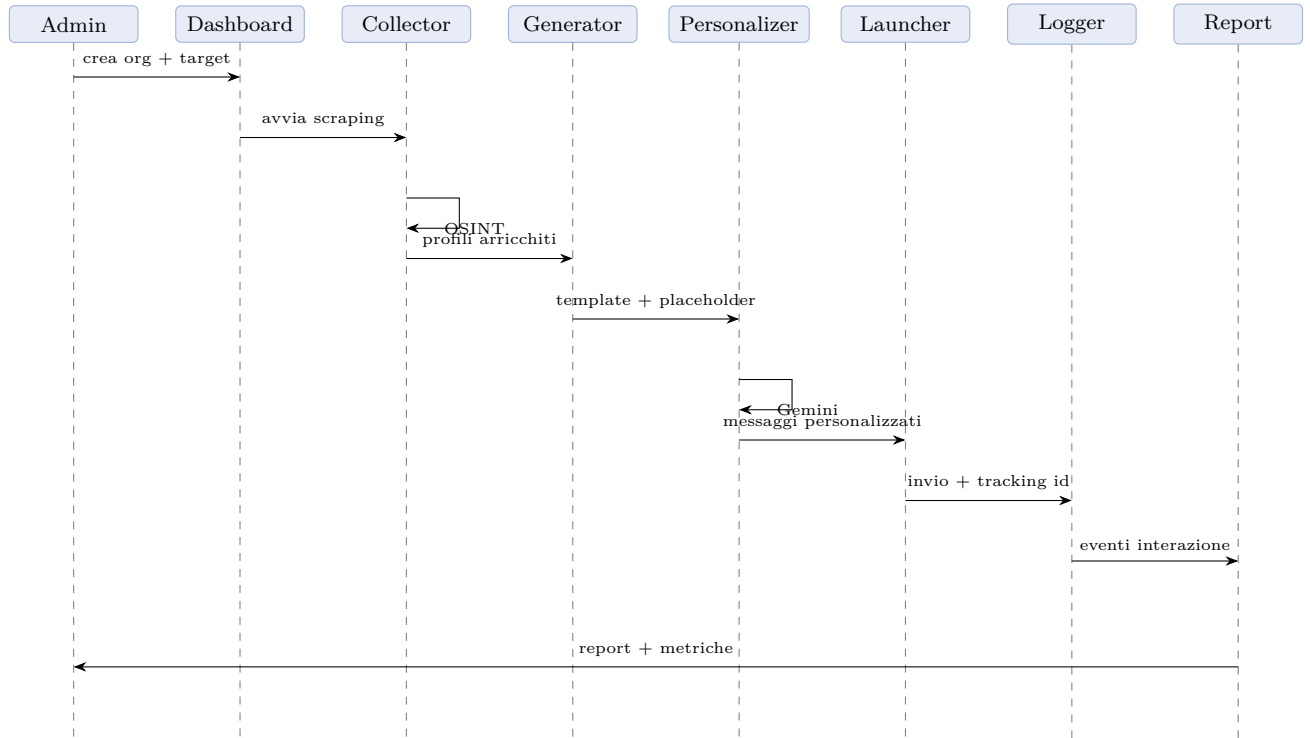


Figura 2.2: Sequence diagram del flusso operativo end-to-end.

2.5 Modello dei dati

Il modello relazionale è riportato in forma sintetica nella Tabella 2.1. Tutte le colonne di chiave esterna sono indicizzate per garantire prestazioni ottimali alle query relazionali. Gli identificatori primari sono UUID (versione 7) generati dal database.

Tabella 2.1: Tabelle principali del modello dati.

Tabella	Descrizione
<code>organization</code>	Organizzazione (tenant) oggetto della simulazione.
<code>organization_member</code>	Associazione tra utente e organizzazione con ruolo (<i>company_admin</i> , <i>analyst</i>).
<code>users</code> + tabelle auth	Utenti e sessioni gestiti da Better Auth.
<code>employee_target</code>	Dipendente-bersaglio (dati base, telefono, sito web, stato scraping sito).
<code>employee_social_profile</code>	Profilo social del target (piattaforma, URL, stato scraping).
<code>employee_scraping_result</code>	Risultato storicizzato di uno scraping (JSON tipizzato).
<code>campaign_template</code>	Template di campagna con configurazione JSON di landing page e allegato.
<code>phishing_campaign</code>	Campagna di phishing (stato: <i>draft</i> , <i>active</i> , <i>completed</i>).
<code>campaign_target</code>	Associazione tra campagna e target con contenuto personalizzato e timestamp di invio.
<code>interaction_log</code>	Evento di interazione con metadati JSON tipizzati per tipo di evento.

3 Scelte Tecnologiche

Lo stack è stato scelto per garantire *type-safety* end-to-end, rapidità di sviluppo e sicurezza in ottica multi-tenant.

3.1 Frontend

- **Nuxt 4** (Vue 3): framework full-stack che combina rendering lato server e SPA, ideale per dashboard interattive e performanti.
- **Nuxt UI v4**: libreria di componenti accessibili con design pulito e coerente per amministratori e analisti.
- **Tailwind CSS v4**: styling *utility-first* per prototipazione rapida e controllo fine del layout.
- **TanStack Query**: gestione dichiarativa di data-fetching, cache e invalidazione lato client.

3.2 Backend

- **Nuxt Server (Nitro)**: layer server-side per esporre endpoint HTTP con architettura full-stack omogenea in TypeScript.
- **Better Auth**: autenticazione moderna (e-mail/password, sessioni via cookie, ID UUID) con gestione di ruoli e controllo degli accessi.
- **Drizzle ORM** (Relational Queries v2): toolkit type-safe per l'accesso al database con schema definito in TypeScript.
- **PostgreSQL**: database relazionale affidabile, con supporto a tipi enumerati e colonne JSONB per i metadati.

3.3 Intelligenza artificiale e scraping

- **Google Gemini** (modello `gemini-2.5-flash`), orchestrato dal framework **Mastra**: genera i messaggi di phishing personalizzati a partire da template e profili OSINT. *Nota*: rispetto all'analisi iniziale, che ipotizzava l'uso di un modello Llama tramite provider esterno, l'implementazione definitiva adotta Google Gemini per stabilità, qualità e disponibilità.
- **rebrowser-puppeteer**: motore di automazione di browser headless per lo scraping controllato di profili LinkedIn e GitHub, simulando la navigazione di un utente reale.
- **cheerio + DNS/WHOIS**: analisi dei siti web personali via richieste HTTP (estrazione di contenuti, link, tecnologie, record DNS e WHOIS), con *guard* anti-SSRF.

3.4 Payload e simulazione di compromissione

Per simulare scenari di compromissione tramite allegati in ambiente controllato è prevista l'integrazione con un framework di *command and control* (Sliver): all'apertura dell'allegato, il payload effettua una *callback* verso il backend, permettendo di tracciare quali utenti hanno eseguito il file senza causare alcun danno reale.

3.5 DevOps e qualità del codice

- **pnpm**: package manager efficiente per la gestione delle dipendenze.
- **Docker Compose**: orchestrazione dell'infrastruttura di sviluppo (PostgreSQL, storage S3-compatibile, server SMTP di test).
- **Drizzle Kit**: gestione delle migrazioni, applicate automaticamente all'avvio.
- **TypeScript**: linguaggio principale, per type-safety end-to-end.

4 Implementazione

4.1 Architettura a livelli del backend

Il backend segue un'architettura a livelli con responsabilità nette, in cui ogni livello invoca solo quello sottostante (Figura 4.1):

- **api/**: livello degli *handler* HTTP, responsabile di validazione (Zod), autenticazione, autorizzazione e instradamento verso i servizi;
- **services/**: logica di business, ossia regole di dominio, orchestrazione ed effetti collaterali;
- **lib/**: configurazione delle librerie esterne (funzioni `setupXXX`);
- **utils/**: i *composable* `useXXX` che forniscono le istanze infrastrutturali con caching pigro.

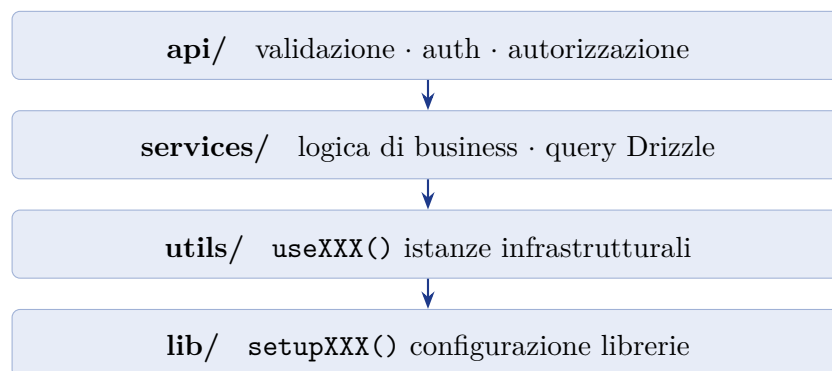


Figura 4.1: Architettura a livelli del backend.

4.2 Autenticazione e multi-tenancy

L'autenticazione è delegata a Better Auth (e-mail/password con *auto sign-in*). Ogni *handler* scoped su organizzazione recupera l'appartenenza del chiamante, esegue una *ability* di autorizzazione e poi invoca un servizio la cui query è vincolata all'`organizationId`: in questo modo l'isolamento tra tenant è garantito e gli accessi cross-organizzazione sono negati. Alla creazione di un'organizzazione, il creatore diventa *company_admin*, vengono inseriti quattro template di default e caricati i relativi allegati su storage S3.

4.3 Target Information Collector (OSINT)

Il modulo dati distingue `employee_target` (persona e campi HR, con un blocco denormalizzato per il sito web) da `employee_social_profile` (profilo social con stato

di scraping) e `employee_scraping_result` (storico dei risultati in JSON tipizzato). L'attivazione dello scraping non avviene in linea: l'endpoint marca i profili come *pending*, mentre un *task* schedulato ogni 10 secondi processa la coda.

Lo scraper LinkedIn/GitHub è basato su `rebrowser-puppeteer` e utilizza un'istanza di Chrome headless con eventuale iniezione di cookie di sessione. Lo scraping dei siti web usa invece `fetch+cheerio` con *guard* anti-SSRF (blocco di URL con credenziali, host locali e indirizzi IP privati/loopback), estrazione di titolo, descrizione, sezioni, link classificati (e-mail, GitHub, LinkedIn), *fingerprint* tecnologico e record DNS/WHOIS.

```
interface GitHubScrapingData {
  platform: 'github'
  username: string | null
  fullName: string | null
  company: string | null
  location: string | null
  followers: number | null
  repositories: GitHubRepoScraped[] // nome, linguaggio, stelle, README,
    contributori
  pinnedRepos: string[]
}
```

Listing 4.1: Struttura tipizzata di un risultato di scraping GitHub.

4.4 Phishing Campaign Generator

I template sono memorizzati nella tabella `campaign_template` con due configurazioni JSON tipizzate: `landingPageConfig` (titolo, colore, campi del form, URL di redirect) e `attachmentConfig` (nome file, tipo, percorso su storage). Ogni organizzazione riceve quattro template predefiniti: *Password Reset*, *Invoice Payment* (con allegato PDF), *CEO Urgent Request* e *SMS Verification*. Il corpo dei messaggi supporta *placeholder* con doppie parentesi graffe, che possono essere sia riferimenti a campi noti sia istruzioni in linguaggio naturale per il modello.

4.5 NLP Text Personalizer

La personalizzazione è affidata a un *agent* Mastra configurato sul modello `gemini-2.5-flash` tramite il *provider* nativo Google. Al lancio, per ciascun target vengono compilati i dati OSINT (LinkedIn, GitHub, sito) in un blocco testuale e inseriti, insieme a template e metadati HR, in un *prompt* specifico per canale (e-mail o SMS). Il prompt contiene regole dettagliate su come interpretare l'OSINT (distinguere il ruolo attuale dai progetti personali, usare l'OSINT per scegliere dettagli credibili e non come argomento del messaggio) e vincola l'output a un oggetto JSON.

```
// Il bridge della chiave verso GOOGLE_GENERATIVE_AI_API_KEY consente al
// provider nativo di autenticarsi; l'uso della stringa "google/<model>"
// evita l'instradamento verso il gateway (che, privo di chiave, si blocca).
model: 'google/gemini-2.5-flash'
```

Listing 4.2: Configurazione del *provider* nativo Google (estratto).

4.6 Campaign Launcher

Il lancio imposta immediatamente lo stato della campagna a *active* e avvia un *task* di background che, per ciascun target: recupera l'OSINT, invoca la personalizzazione, salva il contenuto personalizzato, assembla il messaggio finale iniettando il tracciamento (pixel e link con UUID) e l'eventuale allegato PDF, e infine invia. Il canale e-mail usa SMTP, mentre il canale SMS produce una simulazione su console (nessun gateway reale).

4.7 Interaction Logger

Tutti gli eventi convergono in un unico servizio `logInteraction` che li persiste nella tabella `interaction_log` con metadati JSON tipizzati. Gli endpoint di tracciamento sono: pixel di apertura (GET `/api/track/pixel/:id`), click con redirect (GET `/api/track/click/:id`), invio del form (POST `/api/track/submit/:id`) e callback dell'allegato (POST `/api/track/attachment/:id`). La pagina di login simulata è servita da `/lp/:id`. Per coerenza, gli eventi vengono ignorati se la campagna è già *completed*.

I tipi di evento tracciati sono riportati nella Tabella 4.1.

Tabella 4.1: Tipi di evento di interazione.

Evento	Canale	Descrizione
email_opened	E-mail	Caricamento del pixel di tracciamento.
link_clicked	E-mail/SMS	Click sul link con UUID.
form_submitted	E-mail/SMS	Invio credenziali sulla landing page.
payload_executed	E-mail	Callback dell'allegato con payload.

4.8 Risk Report Generator e Dashboard

La dashboard di dettaglio della campagna calcola in tempo reale gli indicatori di engagement (aperture, click, submission, esecuzione payload, CTR) e mostra un registro live delle interazioni con dettaglio per evento, incluse le credenziali catturate. È inoltre disponibile l'esportazione di un *report di rischio* in formato HTML/PDF,

che include un punteggio sintetico di suscettibilità, la distribuzione della vulnerabilità per dipartimento, l'elenco degli individui ad alto rischio, la *timeline* degli eventi e raccomandazioni formative generate dinamicamente in base ai risultati.

5 Testing e Validazione

5.1 Ambiente di test

La validazione è stata condotta in locale con l'infrastruttura Docker di sviluppo (PostgreSQL, storage S3-compatibile, server SMTP di test *maildev*) e con una chiave Google Gemini valida. Le campagne e-mail sono quindi verificabili end-to-end senza servizi esterni a pagamento. Lo scraping LinkedIn (che richiede cookie di sessione aggiornati) è stato escluso dai test automatici in quanto dipendente da credenziali esterne.

5.2 Test end-to-end della campagna e-mail

È stato eseguito l'intero flusso, dalla registrazione fino al report, guidando l'applicazione tramite un browser reale. Il percorso verificato è:

1. registrazione utente e login (con verifica dei *guard* di rotta);
2. creazione dell'organizzazione (con seeding automatico dei quattro template e caricamento dell'allegato PDF su storage);
3. inserimento di un target (ruolo: *IT Infrastructure Manager*);
4. scraping OSINT del profilo GitHub del target (completato con successo);
5. creazione e lancio di una campagna *Password Reset*;
6. generazione del messaggio personalizzato tramite Gemini e consegna via SMTP;
7. apertura dell'e-mail (attivazione del pixel), click sul link, atterraggio sulla landing page e invio delle credenziali;
8. visualizzazione delle metriche e delle credenziali catturate nella dashboard.

Il tracciamento è stato inoltre validato in modo *realistico*: il rendering dell'e-mail nel client di posta ha attivato autonomamente il pixel (evento `email_opened` registrato con *User-Agent* reale del browser), il click sul link contenuto nell'e-mail ha generato `link_clicked` e l'invio del form ha prodotto `form_submitted` con le credenziali catturate.

5.3 Test dei singoli moduli e casi di sicurezza

Oltre al flusso principale sono stati verificati: la creazione/modifica/cancellazione dei template (inclusa la corretta rimozione di landing page e allegato in fase di modifica), il lancio di una campagna con allegato PDF, la simulazione SMS, la cancellazione delle campagne (consentita solo in stato *draft*), la gestione dei membri, l'interruzione del logging alla chiusura della campagna e l'isolamento multi-tenant. In particolare, il test di sicurezza sull'endpoint di lettura delle interazioni ha confermato che una richiesta con una campagna non appartenente all'organizzazione restituisce *404*, prevenendo la lettura cross-tenant.

5.4 Riepilogo dei risultati

La Tabella 5.1 riassume l'esito dei test condotti.

Tabella 5.1: Riepilogo della campagna di test.

Funzionalità	Esito	Note
Autenticazione e guard di rotta	Superato	register/login/logout
Multi-tenancy e isolamento (IDOR)	Superato	cross-tenant → 404
Creazione org + seeding template	Superato	4 template, PDF su S3
Scraping OSINT GitHub	Superato	profilo completo
Scraping OSINT sito web	Superato	tecnologie, DNS, WHOIS
Template CRUD + rimozione sezioni	Superato	fix rimozione landing page
Personalizzazione AI (Gemini)	Superato	circa 10-15s per target
Consegna e-mail (SMTP)	Superato	recapito su maildev
Campagna con allegato PDF	Superato	PDF allegato (~49 KB)
Tracciamento apertura (pixel)	Superato	UA reale del browser
Tracciamento click	Superato	redirect a landing page
Cattura credenziali (form)	Superato	dati mostrati in dashboard
Callback allegato (payload)	Superato	solo via richiesta diretta
Simulazione SMS	Superato	output su console
Report di rischio (HTML/PDF)	Superato	metriche, timeline, raccomandazioni
Scraping LinkedIn	Non testato	richiede cookie di sessione

6 Valutazione dei Risultati

Il sistema calcola le metriche di efficacia direttamente dai dati di interazione. Le principali sono:

$$\text{CTR} = \frac{\text{click}}{\text{e-mail inviate}} \times 100 \quad \text{Submission rate} = \frac{\text{credenziali inviate}}{\text{click}} \times 100$$

Nel test end-to-end, con un singolo target che ha completato l'intero percorso, la dashboard ha riportato tasso di apertura, click e submission pari al 100%, con il target correttamente classificato come individuo ad alto rischio (*Critical*) nel report generato.

Sul piano qualitativo, i messaggi prodotti da Gemini si sono rivelati credibili e coerenti con il contesto lavorativo del bersaglio: i *placeholder* sono stati risolti con valori reali, i blocchi in linguaggio naturale sono stati trasformati in dettagli concreti (numeri di fattura, fornitori plausibili, scadenze) e il tono è risultato appropriato allo scenario. Rispetto a un template generico, la personalizzazione basata su OSINT aumenta significativamente la verosimiglianza del messaggio e, di conseguenza, la validità della simulazione come strumento di misura della consapevolezza.

7 Limitazioni e Sviluppi Futuri

Coerentemente con la riduzione del gruppo di lavoro, alcune funzionalità previste in fase di analisi sono state ridotte di portata o rimandate. Si riportano di seguito le limitazioni note, distinte per criticità.

7.1 Funzionalità non implementate (sviluppi futuri)

- **Google Dorking:** la ricerca di documenti esposti tramite *dork* non è implementata.
- **Fonti social aggiuntive:** Twitter, Facebook e Instagram sono previsti nell'interfaccia ma non vengono effettivamente processati dallo scraper.
- **Entity resolution:** non è presente una riconciliazione delle identità tra fonti diverse (fuzzy matching) per unificare i profili.
- **Ciclo di feedback:** i dati di interazione non vengono ancora reimmessi nell'arricchimento dei profili.
- **Ruolo Admin di piattaforma** e flusso di invito via e-mail per utenti non ancora registrati.

7.2 Funzionalità implementate con vincoli

- **Scraping LinkedIn:** funzionante ma dipendente da cookie di sessione aggiornati; in loro assenza può produrre dati non validi.
- **Canale SMS:** attualmente una simulazione su console, senza gateway reale.
- **Payload/allegato:** la callback è un semplice *ping* e si attiva solo con specifici lettori PDF; non raccoglie informazioni sull'host.

7.3 Miglioramenti di prodotto

Tra i miglioramenti previsti: grafici e *heatmap* di vulnerabilità nel report, controllo esplicito del tono (formale/urgente/informale), una dashboard aggregata a livello di organizzazione e l'estensione della copertura dei test automatizzati.

8 Conclusioni

Fortify dimostra la fattibilità di una piattaforma di simulazione di social engineering guidata dall'intelligenza artificiale, in cui la raccolta OSINT automatizzata e la generazione di contenuti personalizzati si combinano per riprodurre in modo etico attacchi di *spear phishing*. I sei componenti principali, l'autenticazione multi-tenant e l'intera catena e-mail (personalizzazione, invio, tracciamento e reportistica) sono stati implementati e validati end-to-end.

Il valore del sistema risiede nella chiusura del ciclo simulazione, misurazione e miglioramento: le metriche e i report prodotti trasformano un attacco simulato in informazione azionabile per la formazione. Gli sviluppi futuri, orientati alla copertura di ulteriori fonti OSINT, alla riconciliazione delle identità e all'arricchimento visuale della reportistica, potranno ampliare ulteriormente l'efficacia della piattaforma.

Bibliografia

- [1] OWASP Foundation, *OWASP Top 10, A07:2021 Identification and Authentication Failures / Phishing*, <https://owasp.org>.
- [2] MITRE, *ATT&CK: Phishing (T1566)*, <https://attack.mitre.org/techniques/T1566/>.
- [3] NIST, *Special Publication 800-115: Technical Guide to Information Security Testing and Assessment*.
- [4] ENISA, *Social Engineering: Threat Landscape and Good Practices*.
- [5] Nuxt, *Nuxt 4 Documentation*, <https://nuxt.com/docs>.
- [6] Drizzle Team, *Drizzle ORM: Relational Queries v2*, <https://orm.drizzle.team>.
- [7] Better Auth, *Documentation*, <https://www.better-auth.com/docs>.
- [8] Mastra, *Agent Framework Documentation*, <https://mastra.ai>.
- [9] Google, *Gemini API Documentation*, <https://ai.google.dev/gemini-api/docs>.
- [10] Puppeteer, *Documentation*, <https://pptr.dev>.
- [11] The PostgreSQL Global Development Group, *PostgreSQL 18 Documentation*.